ES6(ECMA Script)

- ES6 is a *modern subset* of JavaScript that allows for cleaner, more efficient, and more powerful code than previous versions

- ES6 features like arrow functions, promises, and async/await simplify asynchronous JavaScript programming, which is essential for fetching data from web APIs.

Arrow functions:

- **ES6 Arrow functions** enable us to write functions with simpler and shorter syntax and make our code more readable and organised.
- It provides a concise syntax for writing function expressions using (=>) operator.

Example: traditional way of writing function in js

```
function multiply(a, b) {
    return a * b;
}
console.log(multiply(3, 5));
```

o/p=15

Using Arrow Function:

Syntax:

For Single Argument:

let function_name = (argument1) => expression

e.g
```
const square = (x) => {
  return x * x;
};
```

For Multiple Arguments:

let function_name = (argument1, argument2 , ...) => expression

e.g.

```
let value = (a, b) => a * b;
console.log(value(3, 5));
```

o/p=15

Arrow functions with multiple lines

- From the previous example, we can see that when there's a single line of code to be executed we didn't use the return keyword but if there are more than two lines to be processed, we need to use a return keyword.

  Example :

  ```
  Let number = (a, b) => {
     c = 5;
     return (a + b) * c;
  };
  console.log(number(2, 3));
  ```

Arrow function with no parameters:

```
()=>{ expressions}
```

e.g.-

```
const string = () => "welcome";
console.log(string);
```

Nested Arrow functions:

```
const outerFunction = (outerVariable) =>
 {
   const innerVariable = 'welcome';

       const innerFunction = (name) =>
{

  console.log(outerVariable + name + ' ' +
innerVariable);
   };
   innerFunction('John'); // calling inner function
};

outerFunction('Hello, ');
```

o/p: Hello, john welcome

Asynchronous programming:

- Asynchronous JavaScript is a programming technique that allows code to initiate potentially long-running tasks (like network requests or file operations) and continue executing other code without waiting for the task to complete.
- Thus preventing the application from freezing or becoming unresponsive.
- Several methods are used to implement asynchronous behavior in JavaScript.

1. Callbacks:

- This is the traditional method where a function is passed as an argument to an asynchronous function and is executed once the operation completes.

  Note : When you pass a function as an argument, remember not to use parenthesis.

- But callbacks are difficult to write and difficult to debug.
- Because of this, most modern asynchronous JavaScript methods don't use callbacks. Instead, in JavaScript, asynchronous programming is solved using **Promises** instead.

2.Promises:

Basic terminology

- A “producing code” that does something and takes time.
- A “consuming code” that wants the result of the “producing code” once it’s ready.
- A *promise* is a special JavaScript object that links the “producing code” and the “consuming code” together.
- Promises are objects representing the eventual completion (or failure) of an asynchronous operation and its resulting value.

Promise States

A promise can be in one of three states:

- **Pending**: Initial state, the operation is still running.
- **Fulfilled** (or Resolved): The operation completed successfully, and a result value is available.
- **Rejected**: The operation failed, and an error (reason) is available.

Creating a Promise:

You create a promise using the Promise constructor, which takes an "executor" function as an argument

Syntax:

Executor function

```
let promise = new Promise (function(resolve, reject)
{
  // executor (the producing code )
}
);
```

- When new Promise is created, the executor runs automatically. It contains the producing code which should eventually produce the result.

- Its arguments resolve and reject are callbacks provided by JavaScript itself

- When the executor obtains the result, it should call one of these callbacks:

resolve — if the job is finished successfully,
reject — if an error has occurred,

- Then use .then(), .catch(), and .finally() methods to consume its eventual value or error

```
promise.then(
  function(result) { /* handle a successful result */ },
  function(error) { /* handle an error */ }
);
```

- Catch

  If we're interested only in errors.

- finally

  Just like there's a finally clause in a regular try {...} catch {...}

3. **async/await:** built on top of promises, allows developers to write asynchronous

code that looks and behaves like synchronous code.

- An async keyword before a function declaration makes it an asynchronous function that implicitly returns a promise.

- The await keyword, used only inside an async function, pauses the execution of that function (without blocking the main thread) until the promise it's waiting for is resolved

Conclusion:

- **Callbacks:** The traditional approach using functions passed as arguments.
- **Promises:** A better alternative that improves readability and avoids callback nesting.
- **Async/Await:** A modern and cleaner syntax that makes asynchronous code look synchronous.

Feature	Callbacks	Promises	Async/Await
Definition	Functions passed as arguments to other functions.	Objects representing eventual completion or failure of an asynchronous operation.	Syntactic sugar over Promises, using async functions and await expressions.
Readability	Can lead to "callback hell" with nested functions, making code hard to read.	Allows chaining with .then() and .catch(), improving readability.	Provides a more synchronous-like flow, enhancing readability.
Error Handling	Requires manual checks and can be cumbersome in nested callbacks.	Errors are handled using .catch(), centralizing error management.	Utilizes try...catch blocks, simplifying error handling.
Control Flow	Sequential execution, but complex flows can become difficult to manage.	Supports chaining, allowing for more manageable control flows.	Enables writing asynchronous code in a synchronous manner, simplifying control flow.

Exp 4: needs to generate API key

Step-by-Step Guide

1. **Sign Up/Log In**: Go to the OpenWeather registration page and create a new account or sign in to your existing one.
2. **Verify Email**: A confirmation email containing your default API key will be sent to the email address you provided. You must

verify your email address to activate your account and the key.

3. **Navigate to API Keys**: Once logged in, go to your account dashboard and click on the "**API keys**" tab or select "**My API Keys**" from the dropdown menu under your username in the top-right corner.
4. **Generate New Key (Optional)**:

- A default API key is automatically generated upon registration.

5. **Copy Your Key**: Your API key (a 32-character random string) will be displayed. Copy and securely store this key, as you will need it for every API call.